Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Summer, Year:2026), B.Sc. in CSE (Day)

Lab Report NO : 3
Course Title: Artificial Intelligence Lab
Course Code: CSE 316 Section: 241-D2

Lab Experiment Name: Implementation Graph Coloring and Scheduling Problems using CSP and Backtracking Algorithm.

Student Details

	Name	ID
1.	Md. REAHOON ZANNAH	223002038

Lab Date : 27-07-2026
Submission Date : 03-08-2026
Course Teacher's Name : Sian Ashsad

Lab Report Status

Marks: ……. **Signature:.....**
Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Implementation Graph Coloring and Scheduling Problems using CSP and Backtracking Algorithm.

2. OBJECTIVES

- To understand the basic concept of Constraint Satisfaction Problems (CSP).
- To learn how to represent real-world problems, such as map coloring and exam scheduling, using graph data structures (adjacency lists and matrices).
- To implement the Graph Coloring Algorithm using backtracking in Python.
- To find the minimum number of colors or days required to satisfy all given constraints.

3. PROCEDURE

❖ Step of both Problem:

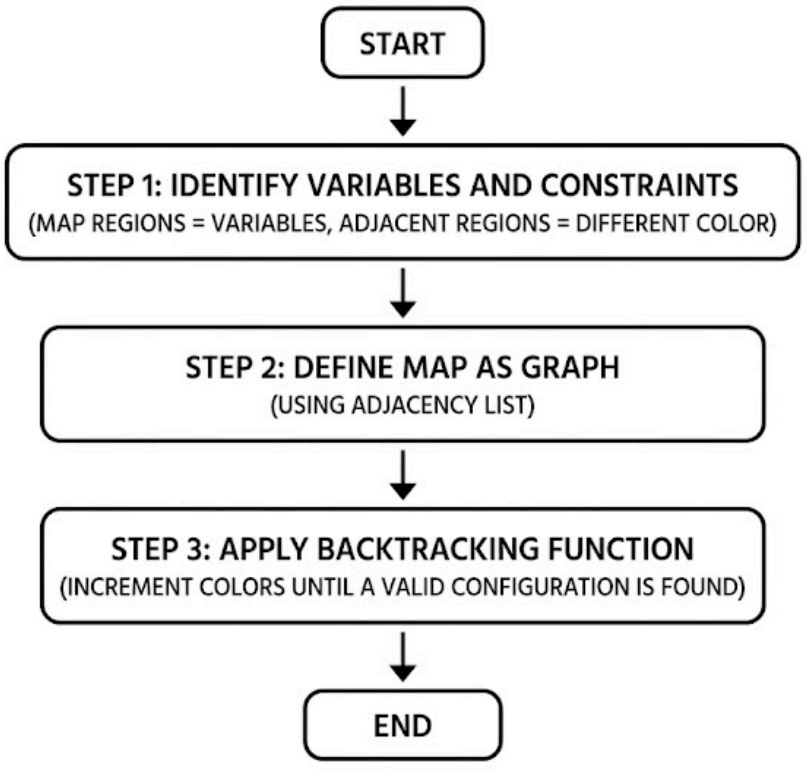

Fig: 3.1: Procedure.

4. IMPLEMENTATION:

a) Graph Coloring Problem:

```
def print_adjacency_matrix(nodes, adj_list):
    print("--- Adjacency Matrix ---\n  " + " ".join(nodes))
    for node in nodes:
        row = [1 if neighbor in adj_list[node] else 0 for neighbor in
nodes]
        print(f"{node} " + " ".join(map(str, row)))
    print()

def is_safe(node, color, adj_list, color_assignment):
    return not any(color_assignment.get(neighbor) == color for neighbor in
adj_list[node])

def graph_coloring(nodes, adj_list, m, color_assignment, node_idx=0):
    if node_idx == len(nodes):
        return True

    current_node = nodes[node_idx]

    for color in range(1, m + 1):
        if is_safe(current_node, color, adj_list, color_assignment):
            color_assignment[current_node] = color

            if graph_coloring(nodes, adj_list, m, color_assignment,
node_idx + 1):
                return True

            color_assignment[current_node] = 0

    return False

nodes = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
adj_list = {
    'A': ['B', 'D', 'E'],
    'B': ['A', 'C', 'D', 'E', 'F'],
    'C': ['B', 'D', 'F'],
    'D': ['A', 'B', 'C', 'E', 'F', 'G'],
    'E': ['A', 'D', 'G', 'B'],
    'F': ['B', 'C', 'D', 'G'],
    'G': ['D', 'E', 'F']
}

print("--- Adjacency List ---")
for node, neighbors in adj_list.items():
    print(f"{node}: {neighbors}")
```

```
print()

print_adjacency_matrix(nodes, adj_list)

color_names = {1: 'Red', 2: 'Green', 3: 'Blue', 4: 'Yellow', 5: 'Orange'}
color_assignment = {node: 0 for node in nodes}

min_colors = 1
while not graph_coloring(nodes, adj_list, min_colors, color_assignment):
    color_assignment = {node: 0 for node in nodes}
    min_colors += 1

print("--- Final Output (Color Assignment) ---")
print(f"Minimum number of colors required: {min_colors}")
for node in nodes:
    print(f"Region {node} -> {color_names[color_assignment[node]]}")
```

b) Scheduling Problems:

```
def print_adjacency_matrix(nodes, adj_list):
    print("--- Adjacency Matrix ---\n  " + "   ".join(nodes))
    for i, node in enumerate(nodes):
        row = [1 if neighbor in adj_list[node] else 0 for neighbor in
nodes]
        print(f"{node}  " + "   ".join(map(str, row)))
    print()

def is_safe(node, day, adj_list, schedule_assignment):
    return not any(schedule_assignment.get(neighbor) == day for neighbor in
adj_list[node])

def schedule_exams(nodes, adj_list, max_days, schedule_assignment,
node_idx=0):
    if node_idx == len(nodes):
        return True

    current_node = nodes[node_idx]

    for day in range(1, max_days + 1):
        if is_safe(current_node, day, adj_list, schedule_assignment):
            schedule_assignment[current_node] = day

            if schedule_exams(nodes, adj_list, max_days,
schedule_assignment, node_idx + 1):
                return True
```

```
            schedule_assignment[current_node] = 0

    return False

nodes = ['C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7', 'C8']
adj_list = {
    'C1': ['C2', 'C5', 'C8'],
    'C2': ['C1', 'C3'],
    'C3': ['C2', 'C4', 'C5'],
    'C4': ['C3', 'C5'],
    'C5': ['C1', 'C3', 'C4', 'C6'],
    'C6': ['C5', 'C7'],
    'C7': ['C6', 'C8'],
    'C8': ['C1', 'C7']
}

print("--- Adjacency List ---")
for node, neighbors in adj_list.items():
    print(f"{node}: {neighbors}")
print()

print_adjacency_matrix(nodes, adj_list)

days_mapping = {1: 'Monday', 2: 'Tuesday', 3: 'Wednesday', 4: 'Thursday',
5: 'Friday'}
schedule_assignment = {node: 0 for node in nodes}

min_days = 1
while min_days <= 5 and not schedule_exams(nodes, adj_list, min_days,
schedule_assignment):
    min_days += 1

print("--- Final Output (Exam Schedule) ---")
if min_days > 5:
    print("Cannot schedule within 5 days!")
else:
    print(f"Minimum number of days required: {min_days}\n")
    for node in nodes:
        print(f"Course {node} ->
{days_mapping[schedule_assignment[node]]}")
```

5. OUTPUT

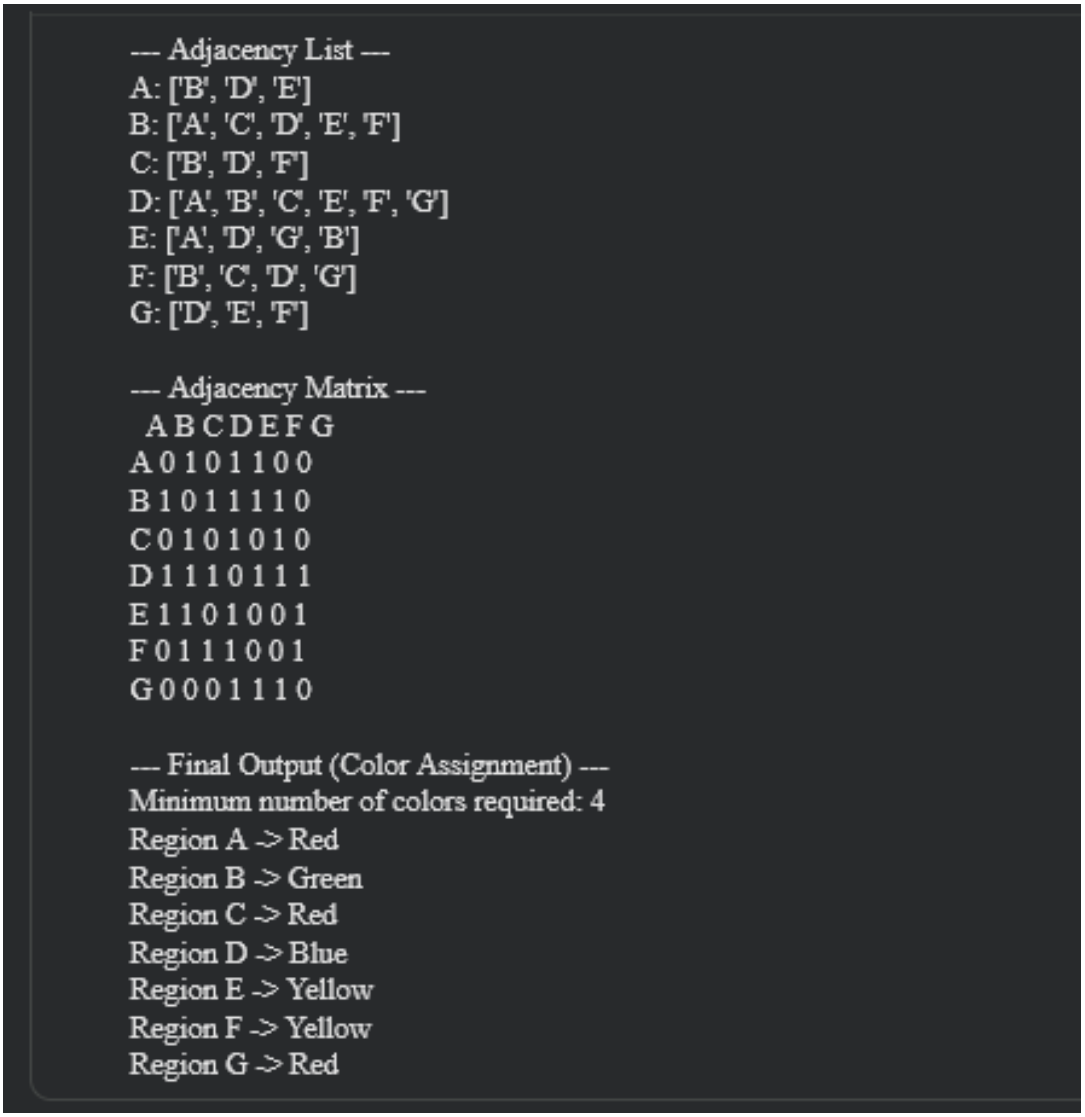

Fig-3.2: Result of (a).

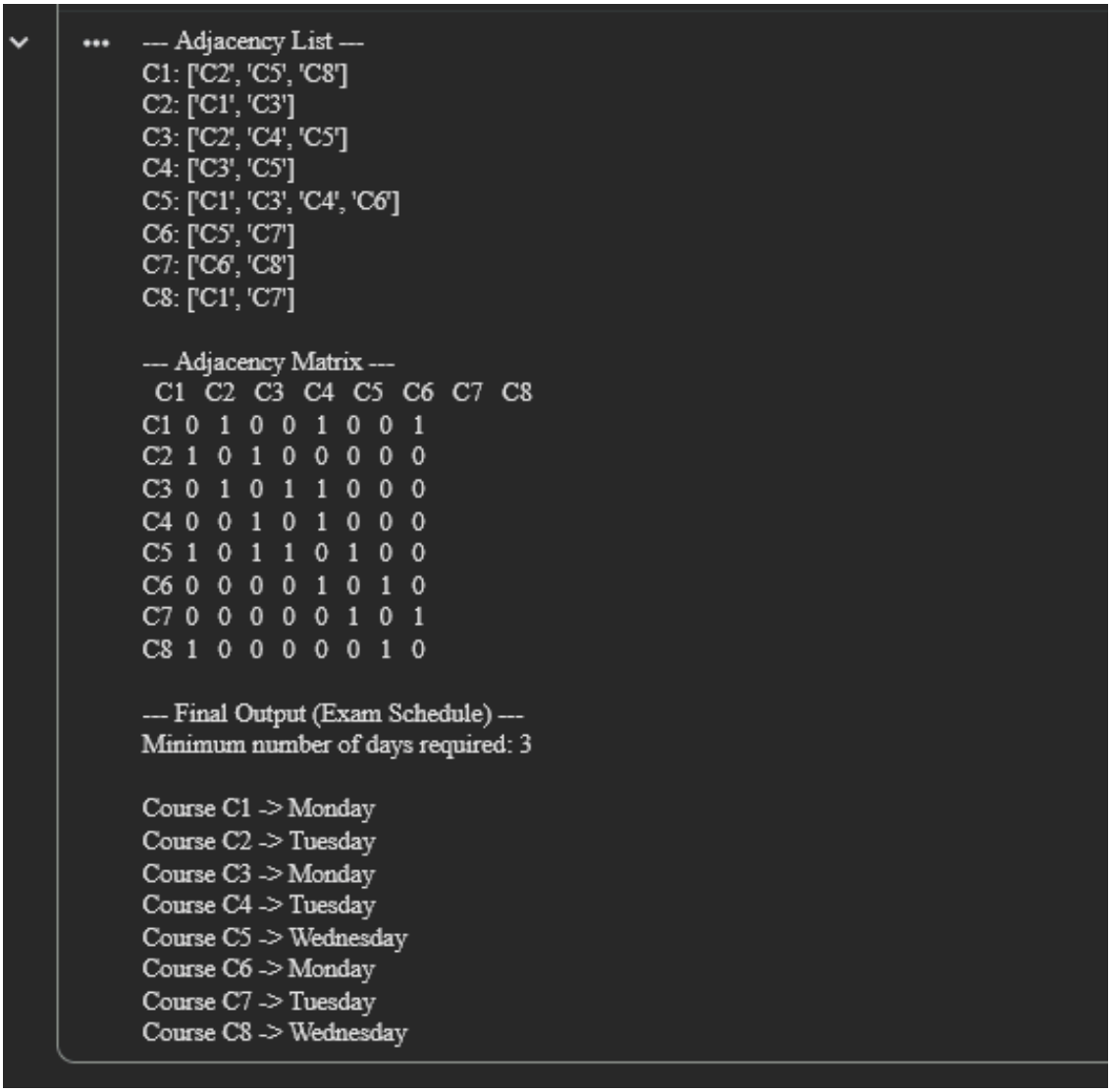

Fig-3.3: Result of (b).

6. ANALYSIS AND DISCUSSION

The backtracking algorithm successfully solved both constraint satisfaction problems by exploring possible assignments and reverting whenever it hit a conflict. In the first problem, the script mapped the country's regions into a graph and determined that at least 4 colors were necessary to ensure no neighboring areas shared the same color. The heavily connected central node (Region D) was the main bottleneck forcing the algorithm to expand its color domain.

In the second problem, the algorithm treated courses as nodes and conflicting students as edges. It processed the schedule sequentially from Monday to Friday. The code found a valid combination using just 3 days (Monday, Tuesday, Wednesday), successfully fitting all 8 exams without any student having to take two exams on the same day. The constraints were rigorously maintained throughout the recursive search.

7. SUMMARY

The backtracking algorithm successfully solved both constraint satisfaction problems by exploring possible assignments and reverting whenever it hit a conflict. In the first problem, the script mapped the country's regions into a graph and determined that at least 4 colors were necessary to ensure no neighboring areas shared the same color. The heavily connected central node (Region D) was the main bottleneck forcing the algorithm to expand its color domain. In the second problem, the algorithm treated courses as nodes and conflicting students as edges. It processed the schedule sequentially from Monday to Friday. The code found a valid combination using just 3 days (Monday, Tuesday, Wednesday), successfully fitting all 8 exams without any student having to take two exams on the same day. The constraints were rigorously maintained throughout the recursive search.

8. GITHUB REPOSITORY:

Link:
https://github.com/pro382r/GUB-Academic/blob/main/AI-lab/lr3/mapColorAndScedeule.ipynb